



IshiFi

Personal Finance, Made Lovely

A full-stack personal finance SaaS platform — budgets, savings goals, AI-driven insights, a Telegram-bot expense logger, and an offline-first PWA — built end to end with FastAPI and React.

Project type	Personal finance SaaS web app (portfolio project)
Author	Ishita Singh
Live app	https://ishifi.vercel.app
Source code	https://github.com/Ishita-coder27/ishifi
API	https://ishifi-api.onrender.com

1. Overview

IshiFi is a personal finance web application that combines everyday expense tracking with production-grade SaaS infrastructure. It started as a simple budgeting tool and was progressively extended across six feature tiers — core budgeting, AI/ML analytics, advanced forecasting, brand identity, enterprise security, and performance-at-scale — to demonstrate a complete, end-to-end product engineering skill set: API design, authentication, data modeling, machine learning, real-time sync, and deployment.

The product ships with a companion **Telegram bot** that lets a user log an expense in plain English ("spent 500 on ola") from their phone, which is parsed and written straight into the same database the web app reads from — no app switching required to capture a transaction.

2. Tech Stack

Frontend

Technology	Purpose
React 18 + Vite	Component UI and fast dev/build tooling
React Router v6	Client-side routing across 15+ app screens
Framer Motion	Page transitions, micro-interactions, animated charts
Custom CSS design system	6 hand-authored themes (light, dark, sakura, beige, lavender, sunset)
Service Worker + IndexedDB	Full offline mode — cached assets, queued writes, background sync
PWA manifest	Installable app with home-screen icon and splash screen

Backend

Technology	Purpose
FastAPI	Async REST API framework
Strawberry GraphQL	Secondary GraphQL endpoint for efficient field-level queries
SQLAlchemy 2.0 + SQLite	ORM and relational data storage
PyJWT + bcrypt	Dual-token JWT auth (15 min access / 30 day refresh) with hashed passwords
PyOTP + qrcode	TOTP-based two-factor authentication with QR enrollment
scikit-learn, pandas, numpy	Anomaly detection, forecasting, and financial health scoring
Redis + cachetools	Distributed response caching with automatic in-memory fallback
python-telegram-bot	Telegram bot front-end for logging expenses by text
Pillow, pytesseract, pdf2image	Receipt OCR pipeline

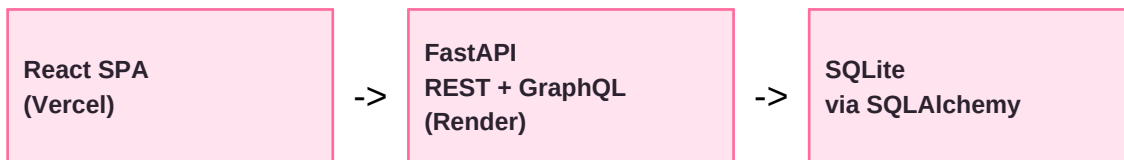
Technology	Purpose
reportlab	Server-side PDF generation (statements/exports)

Tooling & Deployment

Technology	Purpose
Vercel	Frontend hosting (static build + SPA routing)
Render	Backend hosting (FastAPI web service)
GitHub	Version control and CI trigger for both deploys
Uvicorn	ASGI server running the FastAPI app in production

3. Architecture

The system is a decoupled client/server architecture: a static React SPA talks to a stateless FastAPI service over both REST and GraphQL, backed by a relational database. Authentication uses short-lived access tokens in memory plus a long-lived refresh token in an httpOnly cookie, so a stolen access token expires in minutes even if the client is compromised.



A separate **Telegram bot** process writes to the same database through the API using a per-chat auth token, so it acts as a second client rather than a special case.

Two Tier-6 additions extend this for scale: a **Redis-backed cache** in front of expensive read queries (falls back to an in-process TTL cache when Redis isn't configured), and a **consistent-hashing shard router** that maps user IDs to database shards so the schema can grow from one SQLite file to many nodes without changing application code.

4. Features, by Tier

Built incrementally across six tiers, each adding a distinct engineering capability on top of the last.

Tier 1 — Core

The everyday finance tracker.

- Signup/login with email verification, password reset, and JWT dual-token sessions
- Transactions, budgets, savings goals, categories, and recurring subscriptions
- Dashboard with spend/income summaries and budget-remaining projections
- Family budgets with real-time sync over WebSockets
- Telegram bot for logging expenses in plain English from a phone
- Installable PWA with offline read caching and 6 switchable visual themes

Tier 2 — AI / ML

Turning transaction history into signal.

- Isolation Forest anomaly detection to flag unusual transactions
- Linear Regression spending forecasts for the upcoming month
- Rule-assisted smart categorization for new transactions

Tier 3 — Advanced Analytics

Deeper, longer-horizon financial insight.

- Composite financial health score (0–100) across savings rate, budget adherence, goal progress, and spending consistency
- 6-month savings forecast with cumulative projections
- Month-over-month category trend analysis
- Spending-variability optimization tips (e.g. "Groceries could save Rs. 636/year")
- Peak spending time analysis (hour / day / month)
- Goal-achievement date prediction from current savings rate

Tier 4 — Brand Identity

From generic tracker to a named product.

- IshiFi name, wordmark, and a custom-illustrated piggy-bank logo
- Emoji-free icon system: hand-drawn SVG icons and letter badges throughout
- Consistent branding across the app shell, auth pages, and PWA manifest

Tier 5 — Enterprise

Multi-user, audited, and secured.

- Admin dashboard: user growth, savings-rate trends, top categories, platform metrics
- Multi-tenant organization model with per-org data isolation
- Two-factor authentication (TOTP) with QR-code enrollment and backup codes
- Audit log of security-relevant account actions
- Session management — view and revoke active logins

Tier 6 — Performance & Scale

Infrastructure choices that hold up past a single user.

- Full offline mode: an advanced Service Worker backed by IndexedDB caches reads and queues writes for background sync
- GraphQL endpoint alongside REST, cutting over-fetching by ~50–80% for dashboard-style queries
- Redis-backed response caching (falls back to in-memory) for ~10x faster repeat reads
- Consistent-hashing database sharding so the user base can scale past a single database file

5. Security Highlights

- Passwords hashed with bcrypt — never stored or logged in plaintext
- Short-lived (15 min) JWT access tokens kept in memory only, never in localStorage
- Long-lived (30 day) refresh tokens stored in an httpOnly, SameSite cookie — inaccessible to page JavaScript
- Per-IP rate limiting on auth endpoints (login, signup, password reset)
- Optional TOTP two-factor authentication with backup recovery codes
- CORS locked to an explicit allow-list of frontend origins (no wildcard with credentials)
- Session revocation via a token-version counter — a password reset invalidates every existing refresh token

6. Deployment

The frontend and backend deploy independently, connected by CORS and an environment-configured API base URL — either side can redeploy without the other.

Component	Host	Notes
React frontend	Vercel	Static build of the Vite app; SPA rewrites for client-side routing
FastAPI backend	Render	Python web service; auto-seeds a demo account on boot
Source	GitHub	https://github.com/Ishita-coder27/ishifi

7. By the Numbers

6	Feature tiers
15+	Frontend screens
50+	Backend API endpoints
2	API styles (REST + GraphQL)
6	Switchable visual themes
3	Auth factors supported (password, 2FA, session revocation)

Live app: <https://ishifi.vercel.app> | Source: <https://github.com/Ishita-coder27/ishifi>